Course Project Specification

Neural Network ; Deep Learning Architecture & Transfer Learning

Academic Level: Master 2
Team Format: Groups of 2–3 Students (Max 3)
Course Context: Programming for AI
Deliverables: Report, Code, Checkpoints, Presentation

1. Project Overview & Objectives

The primary objective of this project is to gain rigorous, hands-on mastery in designing, training, optimizing, and evaluating deep neural networks for computer vision applications. Students will systematically analyze the individual and combined impacts of model depth, kernel size, receptive field configuration, learning rate schedules, optimization algorithms, data augmentation strategies, and transfer learning paradigms (layer freezing vs. fine-tuning).

2. Project Tasks & Technical Requirements

Task 1: Baseline Architecture & End-to-End Pipeline

- **Dataset & Pipeline:** Select a standard multi-class computer vision dataset (e.g., CIFAR-10, CIFAR-100, Tiny ImageNet, or an approved custom dataset) and construct DataLoaders with automated batching and train/test splitting.
- **Baseline Architecture:** Design and implement a multi-layer Convolutional Neural Network (CNN) from scratch without utilizing pre-trained weights.
- **Training Loop:** Train for a sufficient epoch count. Log and record Training Loss, Validation Loss, Validation and Accuracy and per-class accuracy for each epoch.
- **Evaluation & Diagnostics:** Generate combined learning curve plots (Loss & Accuracy across epochs) and calculate the normalized Confusion Matrix on unseen test data.

Task 2: Architectural Variations, Hyperparameters & Augmentation

- **Architectural Tuning:** Vary network depth (add/remove convolutional and linear layers), alter kernel sizes (3x3, 5x5, 7x7), adjust strides and padding, and incorporate Batch Normalization layers.
- **Hyperparameter Search:** Compare multiple optimization algorithms (e.g., SGD with Momentum, Adam, AdamW), experiment with initial learning rates (1e-2, 1e-3, 1e-4), and evaluate learning rate schedulers.
- **Data Augmentation:** Implement comprehensive data pipelines incorporating random flips, affine transformations, color jitter, cropping, and advanced techniques (MixUp or CutMix).

- **Comparative Analysis:** Compile a comparative benchmark table recording total parameters, runtime latency, peak validation accuracy, and generalization gap.

Task 3: Transfer Learning, Layer Freezing & Fine-Tuning

- **Feature Extraction (Freezing):** Import a modern pre-trained backbone (e.g., ResNet-18, ResNet-50, EfficientNet, or ConvNeXt pre-trained on ImageNet). Freeze all feature extractor layers (requires_grad = False) and train only a custom classification head.
- **Fine-Tuning:** Unfreeze selected upper convolutional blocks (or the entire backbone) and train using a reduced learning rate to fine-tune pre-existing filters to the target dataset.
- **Comparative Evaluation:** Provide detailed empirical comparisons between scratch-built CNNs, frozen feature extractors, and fine-tuned models regarding data efficiency, convergence speed, and final test set metric scores.

📌 DEFENSE & LIVE CODE EXECUTION REQUIREMENTS

• **Mandatory Live Jupyter Notebook Execution**: During the oral defense, each group must open and execute their Jupyter Notebook live in front of the evaluators. All code cells must execute seamlessly without runtime errors.

• **Model Checkpoint Loading & On-Site Testing**: All trained model weights (.pt / .pth) MUST be saved. Evaluators will prompt students to dynamically load saved weights on site and perform inference/evaluation on unseen test mini-batches.

• **Individual Oral Defense & Direct Code Assessment**: Every team member **MUST** demonstrate complete mastery of all project code and underlying deep learning theory, regardless of individual task allocation.

• **On-the-Spot Live Code Modifications**: Students will be asked to modify specific lines of code live (e.g., changing hyperparameter values, modifying activation functions, adjusting custom data transforms, or altering loss formulations) and justify the runtime behavior.

3. Evaluation & Grading Schema

Evaluation Component	Weight	Assessment Criteria
Written Technical Report	**25%**	Methodological rigor, structural comparison tables, learning curves, confusion matrix diagnostics, and depth of empirical analysis.
Code Quality & Architecture	**20%**	Clean modular design, reproducibility, proper PyTorch pipeline implementation, and comprehensive inline code documentation.
Live Execution & Checkpoints	**15%**	Flawless live execution of Jupyter Notebooks and successful on-site loading and inference using saved model checkpoints.
Individual Defense & Live Modification	**40%**	In-depth understanding of all code lines, theoretical grasp of deep learning concepts, and ability to modify code live upon request.

4. Submission Package Guidelines

Submit a zipped archive named [Group_Number_Name1_Name2_Name3].zip containing:

- **1. Technical Report (Report.pdf):** PDF formatted document (6–8 pages) following standard IEEE conference paper format detailing methodology, experiments, and analysis.
- **2. Source Code (Notebooks/):** Fully executed Jupyter Notebooks (.ipynb) containing pre-computed outputs, inline comments, and visualization cells.
- **3. Model Checkpoints (Checkpoints/):** Saved model checkpoint files (.pt, .pth) corresponding to all experimental configurations and tasks.
- **4. Presentation Deck (Presentation.pdf):** Presentation deck detailing problem formulation, comparative results tables, learning curves, and key findings.
- **Each student must individually upload their .zip file to the Teams Class platform under the assignment tab, even though the work is performed in groups.**